

Rajendran Prathibha Devkar

Dallas, TX • (682) 406-5414 • prathibhadevkar@gmail.com • linkedin.com/in/prathibha-devkar-3096

Software Engineer (4+ years) shipping production full-stack apps and ML systems for enterprise clients — React, React Native, Node.js, PyTorch — backed by independent research into applied ML systems.

EDUCATION

Master of Science in Computer Science — The University of Texas at Arlington, USA May 2026
Bachelor of Technology in Computer Science — Saintgits College of Engineering, India May 2019

EXPERIENCE

Research Assistant, SEAR Lab — The University of Texas at Arlington Apr 2026 - Present

- Refactored a 6-service Docker stack (MQTT, InfluxDB, Grafana, a FastAPI Data Hub, Python collectors) into a single self-contained FastAPI app with an embedded dashboard, eliminating the Docker/Grafana/MQTT dependency entirely.
- Integrated live telemetry from three hardware APIs — LG ThinQ (AC), Rheem EcoNet (water heater), Enphase (solar) — into a unified REST API computing a real-time Net Load metric, via a background asyncio collector refreshing every 60 seconds.

Software Developer Intern, Board Americas — Remote, Texas May 2025 - Mar 2026

- Resolved 45+ medium-to-high severity production bugs over 11 months across an enterprise forecasting platform (Aurelia/TypeScript, C#), improving stability across UI, state management, and backend API layers.
- Diagnosed and fixed critical D3.js bugs — fiscal-year x-axis errors and incorrect actual-vs-forecast rendering clients relied on.
- Fixed a company-wide cache-invalidation bug (per-user to per-company scope) that let forecast data drift out of sync, and a null-date bug across all datepickers that was triggering 500 errors in production.

Technology Analyst, Infosys Private Ltd. — Trivandrum, India Oct 2019 - Nov 2022

- Developed React Native UI and integrated backend APIs for a curbside-pickup feature on the Tractor Supply Company mobile app, improving user satisfaction by 30% based on pilot-release feedback.
- Built RESTful APIs in Node.js for an e-learning platform supporting 200+ concurrent users across student, teacher, and admin roles, enabling real-time scheduling, participation, and registration.
- Designed PostgreSQL schemas and built a JMeter load-testing pipeline (1K-10K virtual users) that identified and resolved a calendar API bottleneck, for a team-reported 40% improvement in data-retrieval speed.

SKILLS

Databases: SQL Server, PostgreSQL, MySQL

Frontend: React, React Native, Aurelia, D3.js, HTML5, CSS3

Languages: Python, TypeScript, JavaScript, C#

ML: PyTorch, TensorFlow, Keras, OpenCV, NumPy, Pandas, CNN

AI/LLM: Ollama, Prompt Engineering, DistilBERT Fine-Tuning, Confidence Calibration

Backend / APIs: ASP.NET, Node.js, FastAPI, RESTful APIs

Cloud: AWS, GCP

DevOps / Tools: Azure DevOps, Git/GitHub, Jira, Docker

PROJECTS

GlassBox — OpenTelemetry, FastAPI, React, MCP

- Built GlassBox, an OpenTelemetry observability tool (FastAPI + React viewer) for AI agents; used it to diagnose RDT's real failure — a repetition collapse, not a parser bug — and built a classifier reusing confidence-referee's GroundingSignal unmodified (33% recall, honestly disclosed).

Apex F1 Predictor — Python, FastAPI, scikit-learn, WebSocket

- Built Apex F1 Predictor, a full-stack ML app forecasting F1 race outcomes from historical data (2023+) with a RandomForestClassifier and FastAPI backend (autonomous retraining, WebSocket live telemetry); reached AUC of 0.97/0.91/0.82 for win/podium/points-finish, though exact finishing-position is harder (~4-position MAE) — surfaced directly in the app rather than hidden.

Recurrent Depth Transformer (RDT) — PyTorch, Python

- Implemented a Recurrent Depth Transformer from scratch in PyTorch, replacing stacked transformer layers with a single recurrently-looped block; matched a parameter-matched GPT baseline's accuracy on multi-hop reasoning at 30-47% fewer parameters, then ran inference-time loop-scaling experiments (6/10/14 loops) to test generalization.
- Built a tool-calling agent from the fine-tuned model; caught a misleading eval metric reporting 100% task completion regardless of correctness, verified only 1-of-4 answers were actually correct, and rewrote the harness to report results honestly.

Confidence-Aware Meeting Intelligence — Python, Ollama, DistilBERT, Streamlit

- Built a confidence-aware meeting summarizer/action-item extractor — unlike Otter.ai/Fireflies-style tools (60-90% accuracy, no confidence signal) — combining a local LLM (Ollama, llama3.2:3b) extraction arm with an independent fine-tuned DistilBERT classifier; calibrated a confidence score (logistic regression on quote-grounding, owner-attribution, cross-model agreement, self-reported confidence) on 228 hand-labeled examples, reaching 58% held-out precision for high-confidence items vs. 16% for low-confidence.
- Extracted the confidence-calibration logic into confidence-referee, a standalone MIT-licensed Python package (13 passing tests, CI on Python 3.9-3.12); validated it generalizes to a second domain — RAG-answer hallucination detection on HaluEval — reaching 0.97 AUC and 94% held-out accuracy using one signal, unmodified.

PUBLICATION

Devkar, R. Prathibha, Daniel, M.S., John, N.P., et al. (2021). **Speaking Mouth System for Speech-Impaired People using Hand Gestures**

Built a custom 4,000-image dataset and a 2-layer CNN (96% test accuracy) for an end-to-end gesture-to-speech inference pipeline; published in Springer FSST 2019.